

Branch-and-Cut Redistricting with Auditable Solver Reports

Gio Della-Libera

Draft — May 2026

Abstract

This paper describes the BISECT branch-and-cut implementation contract for re-districting. It is an exact-optimization implementation paper: it separates formulation, separation metadata, solver status, fallback behavior, and RPLAN/RCTX audit lineage so exact claims require proof or bound metadata. It treats fallback and timeout reports as first-class outputs rather than failures to be hidden behind a valid final plan.

1 Introduction

U.16 documents the `bisect-ilp` and `bisect-cli` branch-and-cut surface: `--structure ilp --ilp-method branch-and-cut`. Its claim is not that every redistricting instance is solved exactly at scale, but that exact-optimization runs should expose formulation, separation, proof status, and audit lineage in a reproducible form.

The paper sits between the algorithm-selection vocabulary of U.13 and the fixed-point audit contract of U.20. U.13 asks when an exact method is the right tool; U.16 says what evidence an exact branch-and-cut run must emit before it is allowed to use exact language; U.20 explains how the resulting plan and context become a stable auditable object. The central distinction is that an RPLAN/RCTX certificate can show that a final plan is valid and traceable, while only a solver report can show that a particular model was solved, bounded, or left unresolved.

2 Implementation Contract

The implementation contract distinguishes formulation-only paths, active branch-and-cut paths, solver fallback paths, and completed proof/bound paths. Each run summary should name the model, objective, constraint profile, separation routine status, solve status, incumbent plan source, and any proof or bound metadata.

The report is the boundary between implementation success and exactness. A run can be successful as software because it produced a valid audited plan, while still failing to support an exact optimization claim because the solver timed out, fell back, or never produced a

bound. U.16 therefore requires every output to carry a status label before any exactness language is used.

The atlas framing makes the separated candidate concrete. A candidate incumbent must be inspectable as a graph assignment; if a district appears in disconnected components, the separation routine emits the violated connectivity cut and the model is resolved. The paper’s exactness language therefore belongs to the solver report, while final-plan validity belongs to the RPLAN/RCTX certificate.

For example, on a five-vertex path with two districts, the assignment

$$[0, 1, 0, 1, 1]$$

places district 0 on the separated vertices 0 and 2, with vertex 1 assigned to the other district between them. A separation round does not describe this as “probably bad” or silently repair it. It emits a structured round: schema **branch-and-cut-separation-round-v1**, status **violated-connectivity-cuts**, cut count 4, and disconnected districts [0,1]. The next solve is then performed under the added cuts. If the process ends with a proof, the run may claim a model-relative exact result; if it ends by timeout or fallback, the same final plan may still be auditable but the exactness claim is withheld.

Field	Purpose	Failure mode avoided
Formulation id	Names variables and constraints	Ambiguous model claim
Cut summary	Records separation activity	Hidden non-branch-and-cut path
Solver status	Distinguishes solved, timed out, fallback	False exactness
Proof/bound	Carries exact evidence when available	Valid plan mistaken for proof

Table 1: Minimum branch-and-cut report fields.

Artifact field	Current value family	Interpretation
Report schema	ilp-solve-report-v1	Stable JSON envelope for solver evidence
Model format	cplex-lp	Rebuildable model serialization
Model digest	SHA-256 digest	Tamper check for the LP file
Proof status	proved-optimal, bounded-gap, timed-out, not-solved	Exactness vocabulary
Fallback flag	true/false	Whether a non-ILP plan source was needed
Search strategy	e.g. k2-branch-and-bound	Solver/search family actually used

Table 2: Concrete solve-report fields used by the U.16 implementation.

The package-level lineage repeats the same boundary. A BISECT manifest records the `bisect-ilp-lineage-v1` schema, the ILP solve-report directory, the audit summary path, the audit-summary digest, the fallback flag, and the configured optimality gap. Those fields make the branch-and-cut path observable after the PDF, plan, and context have been copied away from the machine that produced them.

Status	Meaning	Allowed claim
Formulation-only	Model built, no active separation proof	reproducible model contract
Cut-active	Separation ran with cut metadata	branch-and-cut behavior observed
Bounded	Solver reports incumbent and bound	scoped bound under model
Proven	Solver reports proof under assumptions	scoped exact optimum/infeasibility
Fallback	Heuristic or prior plan returned	valid audited fallback only

Table 3: Status vocabulary for branch-and-cut outputs.

3 Audit Artifacts

The final-plan handoff is the RPLAN/RCTX fixed point. A branch-and-cut run can emit a valid RPLAN plus solver report, manifest, and audit certificate. The certificate verifies declared profile and lineage; it does not prove optimality unless the solver report also carries the relevant proof or bound.

The audit package should retain both the plan artifact and the exact-method claim artifact. RPLAN/RCTX records the final assignment and context, while the solver report records whether separation, bounds, proof, timeout, or fallback occurred. Dropping either side creates ambiguity: a valid plan without a solver report is not an exact result, and a solver report without an auditable final plan is not a reusable BISECT output.

The public golden package can be checked either through the RPLAN certificate verifier or through the BISECT manifest verifier.

This is the fixed-point discipline that U.20 later generalizes. The plan and context fix the district assignment and input universe; the manifest fixes the method transcript and sidecar hashes; the ILP summary hash fixes the collection of solve reports. If the LP model, proof-status field, fallback flag, or summary JSON is edited after the fact, the hash chain changes. The verifier does not need to trust the prose claim in this paper; it checks the declared artifacts.

4 Evaluation Plan

The current evidence ladder is fixture and staged exact-contract evidence. Required L0/L1 checks include formulation construction, cut-report emission, fallback status clarity, and

RPLAN sidecar verification. A public U.16 golden package records the branch-and-cut family handoff in the package corpus. A synthetic path8 benchmark package now adds a solved exact branch-and-cut artifact, including the CPLEX-LP model hash, solve report, audit certificate, and verifier manifest. Real-data exact performance and broad runtime claims remain future benchmark evidence.

Claim	Current evidence	Publication gap
Formulation reports are reproducible	L0 formulation fixtures and the path8 benchmark LP/report artifact	larger model corpus
Fallback status is explicit	L0/L1 status tests	failure-mode examples
RPLAN sidecars verify lineage	L1 audit package checks and public golden package plus solved path8 benchmark	larger solver-produced instance
Exact performance scales	future benchmark placeholder	real-data benchmark protocol

Table 4: Evidence ladder for U.16 claims.

Command or package	Evidence produced
Path8 generator example	Builds the public path8 benchmark package with LP artifact, solve report, manifest, RPLAN/RCTX pair, and audit certificate.
RPLAN certificate verifier	Checks that the exported plan and context satisfy the declared certificate profile and lineage expectations.
BISECT manifest verifier	Checks method transcript, ILP audit summary digest, solve-report directory, and manifest-sidecar consistency.

Table 5: Audit commands that define the current U.16 evidence path.

The concrete generator command is:

```
cargo run -p bisect-cli --no-default-features \
  --example u16_branch_cut_path8_benchmark_package
```

The corresponding verification commands target the generated certificate and manifest JSON files.

The path8 package is intentionally small. It is a smoke-test instance for the exact-search contract, not a scaling result. A paper that claims real-data branch-and-cut performance should add solver name and version, hardware, time limit, optimality gap, instance metadata, full transcripts, and archived output directories for every reported instance.

5 Limitations

Exact optimization claims are scoped to the model, data, objective, constraints, and solver status. A valid final plan is not an optimality proof. Large instances may time out or fail

back, and those outcomes must be reported as such rather than normalized away.

The current draft should be read as an implementation-contract paper. It does not yet provide a public real-data solve table, broad runtime evidence, or a claim that branch-and-cut dominates heuristic construction. Those claims require benchmark commands, solver versions, instance metadata, and archived outputs. The included path8 benchmark is enough to test the audit surface, the report schema, and the exactness vocabulary, but it is too small to support claims about court-scale or statewide optimization performance.

6 Conclusion

U.16 turns branch-and-cut from a black-box exact option into an auditable implementation contract. Its contribution is the separation of three claims: the ILP formulation can be rebuilt, the separation loop can report violated connectivity cuts and proof status, and the final plan can be certified as an RPLAN/RCTX fixed point.

Once those semantics are stable, branch-and-cut can sit beside heuristic and construction methods without category confusion: exact claims live in the solver report, validity and lineage claims live in the audit package, and future performance claims live in benchmark evidence.

References

BISECT project. U.16 branch-and-cut redistricting paper plan, 2026. Local planning document and implementation roadmap.